Operating Systems Assignment 8: Memory Management in xv6

Objective

The objective of this assignment is to understand memory management in xv6 by implementing virtual-to-physical address translation, physical memory statistics, page-table inspection, lazy allocation, and the additional memory management mechanisms specified in the assignment.

Implementation and Design

Q1 – Virtual-to-Physical Translation: Implemented `va2pa()` using `walkpgdir()` to translate valid user virtual addresses into physical addresses. The kernel checks page permissions and prints the page directory index, page table index, offset, and P/W/U flags. The user program tests global, local, and dynamically allocated variables, invalid addresses, and parent-child mappings after `fork()`.

Q2 – Physical Memory Statistics: Added a free-page counter in `kalloc.c`, synchronized using `kmem.lock`. The `getmeminfo()`system call reports total pages, free pages, user address-space pages, and page-table pages. Memory usage is checked before and after allocation, process creation, and cleanup.

Q3 – Page-Table Dump: Implemented `vmprint()` to display page directory entries, page table entries, physical addresses, and permission flags. The test checks page-table changes after heap growth and shrinkage and identifies the stack guard page.

Q4 – Lazy Allocation: Modified heap allocation to reserve virtual address space without immediately allocating physical pages. Page faults allocate and map pages on demand. Invalid accesses and allocation failures are handled by terminating the offending process.

Q5–Q8: The remaining mechanisms are documented and tested according to their respective assignment specifications, including memory usage and page-replacement comparisons where applicable.

Challenges and Solutions

Key challenges include preserving page offsets during address translation, maintaining accurate free-page counts, distinguishing present pages from user-accessible pages, and handling unmapped pages during lazy allocation. These are addressed through page-table validation, synchronized allocator updates, and careful handling of page faults and memory cleanup.

Results and Conclusion

The user programs are designed to verify address translation, memory accounting, page-table structure, and demand paging. Memory measurements and page-fault counts should be recorded from actual QEMU executions. The assignment demonstrates how operating systems manage virtual memory, allocate physical frames, enforce permissions, and optimize memory utilization.